Developing a ReAct Sudoku Agent

Suzanne Szabo · U0-HW-08 · Artificial Intelligence, Utah Tech University · October 2026

1. Repository

Git repository: https://github.com/suzanne-szabo/ReAct_Agent_Sudoku_Solver

The repository contains the specifications (`specs/`), the implementation (`sudoku_agent/`), the tests (`tests/`), run and test instructions (`README.md`), the process log (`docs/process-log.md`), and the logs and traces of the live runs (`docs/runs/`).

2. Project Structure and Component Interactions

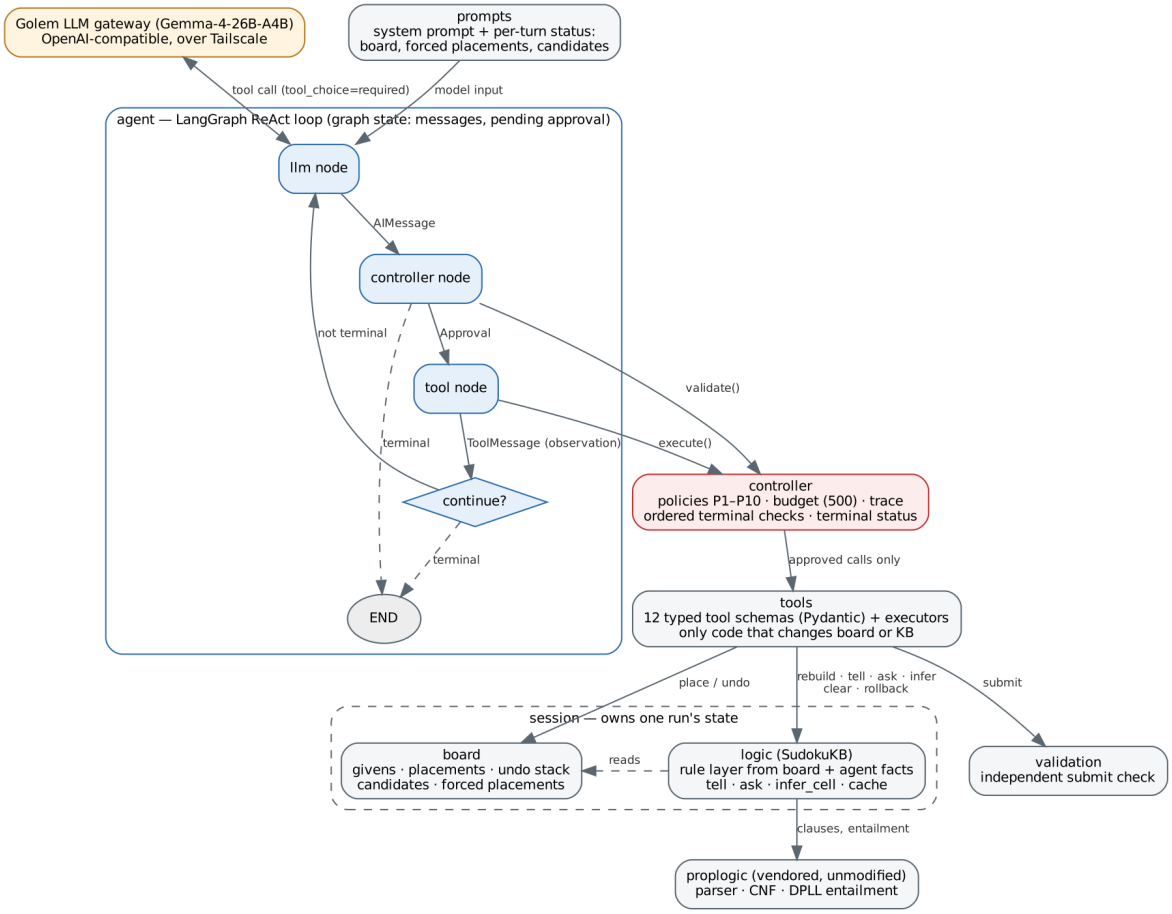


Figure 1. Modules and allowed interactions. The model reaches application state only through the controller (validate, then execute). Only `tools` changes the board or the knowledge base, and only `validation` can establish success.

I designed the structure from my hand-drawn diagram of the loop (LLM, controller, tool, continue?) and then specified it in `specs/architecture.md`. Each turn, the model receives the system prompt, its last 12 turns and a status message rebuilt by the application. It must answer with exactly one typed tool call (`tool_choice="required"`). The controller rejects invalid requests without changing anything, charges one action per call, records

the trace and applies the terminal checks in a fixed order: `SOLVED`, `ACTION_LIMIT`, `REJECTION_LIMIT`, `NO_ACTION`, `MODEL_ERROR`. The knowledge base wraps the course `proplogic` package without modifying it. Its rule layer is generated from the board, so it is satisfiable exactly when the board can still be completed.

3. Development Process

I acted as the orchestrator: I made the architecture and design decisions and reviewed each change. Claude, a coding agent, drafted the specifications, tests and analysis under my direction, and a *separate* coding agent implemented the package from the specifications and tests alone. The 22 commits record each stage in order.

3.1 Specifications before code

Seven specifications define the modules, owned state, interfaces, typed tool schemas, knowledge-base syntax and query meaning, undo behavior, policies, action limits and terminal order. Design choices were measured rather than assumed. A benchmark showed that the full 729-variable Sudoku encoding took up to 22 s per query, so the rule layer covers only the remaining candidates (0.04–0.7 s). When the hardest puzzle still exceeded the 3 s target, I chose to keep the course solver, use no timeouts and relax the targets (decision D16) rather than write a new solver.

3.2 Black-box tests derived from the specifications

Black-box tests were written from the specifications before any implementation existed, and were committed while all of them failed. Expected values such as puzzle solutions, cascades and dead cells were computed by independent scripts. Writing the tests exposed **four underspecified interfaces** (session attributes, puzzle-loader output, `.env` location, forced-cell message format). These were fixed in the specifications before the tests were committed.

3.3 Implementation by a separate coding agent

A fresh coding agent received only the specifications, tests, puzzles and the vendored package, and was not allowed to edit them. It implemented all 13 modules (about 1,900 lines). Its first result was **320 passed, 1 failed**. Instead of working around the failure, it reported two defects in my specifications: one rule put the trace entries for rejected extra calls in the wrong order, and another promised a terminal reason in a trace entry that cannot hold one. In review, I confirmed the module boundaries by searching the code, corrected both specifications, and changed the implementation to match the unchanged test.

3.4 Live runs, failures and revisions

- **First live run (Qwen3.8-27B) failed.** It wasted its first action reading a board it already had, then tried to place a digit in a given cell because it miscounted grid columns. The controller rejected the move (policy P4) without changing anything, and one step took over a minute.

- **I measured before changing anything.** A latency script showed that Gemma-4-26B-A4B answered in 0.6–1.9 s against 7–10 s for Qwen. It also exposed a defect: one call waited 371 s despite the 120 s timeout, because the client library retried twice on its own. I fixed it so the loop's single retry is the only one.
- **Revision v4.** A review of the tool surface found duplicate tools (`get_candidates` duplicated `read_cell`) and descriptions that never said *when* to use a tool. The status message now lists forced placements and every empty cell's candidates, the tools went from 13 to 12, and the prompt became a simple loop: cheapest tools first, then the logic engine. Specifications and tests were updated first (14 tests failed), then the code.

3.5 White-box testing after implementation

A coverage report showed the black-box suite reached 94% branch coverage, with the command-line interface untested. White-box tests written against the code raised coverage to 99%. They found a real defect: a model reply that reused an earlier message id *replaced* that message in LangGraph's state, which crashed the loop. The fix gives every reply a fresh id, and a regression test now covers it.

4. Results

Table 1. Live runs with Gemma-4-26B-A4B (revision v4), 18 October 2026

Puzzle	Status	Actions	Minimum possible	Logic-engine proofs	Rejections
Easy (Wikipedia example, 30 givens)	Solved	52	52	0	0
Hard (Norvig hard1, 17 givens)	Solved	72	65	7	0
Expert (Inkala 2012, 21 givens)	Solved	71	61	10	0

The final suite has 346 passing tests: 298 black-box cases derived from the specifications, 18 white-box tests and the vendored package's 30 tests. Branch coverage is 99%. The actions above the minimum equal the number of logic-engine proofs, so no action was wasted.

5. Takeaway

> **[DRAFT — rewrite in your own words.]** The process change I will keep is writing black-box tests from the specification before any implementation exists. Doing this exposed four vague interfaces before a line of code was written. Later, it let an independent coding agent show me two contradictions in my own specifications instead of quietly coding around them.